Program 1

Objective: Write a program Print Dimensions of dataset.

Code:

```
from sklearn.datasets import load_iris
import pandas as pd

# Load Iris dataset
iris = load_iris()

# Convert to DataFrame
data = pd.DataFrame(iris.data, columns=iris.feature_names)

# Print dimensions
print("Dimensions of the dataset:", data.shape)
print("Number of Rows:", data.shape[0])
print("Number of Columns:", data.shape[1])
```

Output:

```
Dimensions of the dataset: (150, 4)
Number of Rows: 150
Number of Columns: 4
```

Program 2

Objective: Write a program to Calculate of Accuracy Values.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create the model
model = DecisionTreeClassifier()

# Train the model
model.fit(X_train, y_train)

# Predict the test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

Output:

```
Accuracy: 1.0
Accuracy Percentage: 100.0 %
```

Program 3

Objective: Write a program to Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

y = to_categorical(y)

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the ANN
model = Sequential()

# Input layer + Hidden layer
model.add(Dense(10, input_dim=4, activation='relu'))

# Output layer
model.add(Dense(3, activation='softmax'))
```

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=10, batch_size=5)

loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)
```

Output:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential m
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.3417 - loss: 1.3616
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.3500 - loss: 1.2454
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3500 - loss: 1.1452
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3667 - loss: 1.0644
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.4250 - loss: 0.9973
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.4667 - loss: 0.9337
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5000 - loss: 0.8787
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5500 - loss: 0.8289
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5750 - loss: 0.7818
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6500 - loss: 0.7388
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 207ms/step - accuracy: 0.7333 - loss: 0.7310
Test Accuracy: 0.7333333492279053
```

Program 4

Objective: Write a program to Compose Matrix Shape and Tensor Shape.

Code:

```
import tensorflow as tf

# Create a Matrix (2D Tensor)
matrix = tf.constant([
    [1, 2, 3],
    [4, 5, 6]
])

# Create a Tensor (3D Tensor)
tensor = tf.constant([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

# Print Matrix
print("Matrix:")
print(matrix)

# Print Matrix Shape
print("Matrix Shape:", matrix.shape)

# Print Tensor
print("\nTensor:")
print(tensor)

# Print Tensor Shape
print("Tensor Shape:", tensor.shape)
```

Output:

```
Matrix:
tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)
Matrix Shape: (2, 3)

Tensor:
tf.Tensor(
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]], shape=(2, 2, 2), dtype=int32)
Tensor Shape: (2, 2, 2)
```

Program 5

Objective: Write a program to showing Accessing and Manipulation of tensors.

Code:

```
import tensorflow as tf

tensor = tf.constant([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print("Original Tensor:")
print(tensor)

print("\nElement at Row 2, Column 3:", tensor[1][2].numpy())

print("\nFirst Row:")
print(tensor[0].numpy())

print("\nSecond Column:")
print(tensor[:, 1].numpy())

print("\nSliced Tensor:")
print(tensor[0:2, 0:2].numpy())

new_tensor = tensor + 10
print("\nAfter Adding 10:")
print(new_tensor.numpy())

multiply_tensor = tensor * 2
print("\nAfter Multiplication by 2:")
print(multiply_tensor.numpy())

reshaped_tensor = tf.reshape(tensor, (1, 9))
print("\nReshaped Tensor:")
print(reshaped_tensor.numpy())
```

Output:

```
Original Tensor:
tf.Tensor(
[[10 20 30]
 [40 50 60]
 [70 80 90]], shape=(3, 3), dtype=int32)

Element at Row 2, Column 3: 60

First Row:
[10 20 30]

Second Column:
[20 50 80]

Sliced Tensor:
[[10 20]
 [40 50]]

After Adding 10:
[[ 20  30  40]
 [ 50  60  70]
 [ 80  90 100]]

After Multiplication by 2:
[[ 20  40  60]
 [ 80 100 120]
 [140 160 180]]

Reshaped Tensor:
[[10 20 30 40 50 60 70 80 90]]
```

Program 6

Objective: Write a program to showing Accessing and Manipulation of tensors.

Code:

```
import tensorflow as tf

tensor = tf.constant([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print("Original Tensor:")
print(tensor)

print("\nElement at Row 2, Column 3:", tensor[1][2].numpy())

print("\nFirst Row:")
print(tensor[0].numpy())

print("\nSecond Column:")
print(tensor[:, 1].numpy())

print("\nSliced Tensor:")
print(tensor[0:2, 0:2].numpy())

new_tensor = tensor + 10
print("\nAfter Adding 10:")
print(new_tensor.numpy())

multiply_tensor = tensor * 2
print("\nAfter Multiplication by 2:")
print(multiply_tensor.numpy())

reshaped_tensor = tf.reshape(tensor, (1, 9))
print("\nReshaped Tensor:")
print(reshaped_tensor.numpy())
```

Output:

```
Original Tensor:
tf.Tensor(
[[10 20 30]
 [40 50 60]
 [70 80 90]], shape=(3, 3), dtype=int32)

Element at Row 2, Column 3: 60

First Row:
[10 20 30]

Second Column:
[20 50 80]

Sliced Tensor:
[[10 20]
 [40 50]]

After Adding 10:
[[ 20  30  40]
 [ 50  60  70]
 [ 80  90 100]]

After Multiplication by 2:
[[ 20  40  60]
 [ 80 100 120]
 [140 160 180]]

Reshaped Tensor:
[[10 20 30 40 50 60 70 80 90]]
```

Program 7

Objective: Write a program to understand the mechanism of practically training a binary classifier.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load the Breast Cancer dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the Binary Classifier
model = Sequential()

# Hidden Layer
model.add(Dense(16, activation='relu', input_shape=(30,)))

# Output Layer
model.add(Dense(1, activation='sigmoid'))
```

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=16)

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

# Print the accuracy
print("Test Accuracy:", accuracy)

# Predict on test data
predictions = model.predict(X_test)

print("Predicted Values:")
print(predictions[:5])
```

Output:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential m
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
29/29 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8132 - loss: 0.4242
Epoch 2/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9165 - loss: 0.2757
Epoch 3/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9209 - loss: 0.2158
Epoch 4/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9407 - loss: 0.1811
Epoch 5/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9473 - loss: 0.1577
Epoch 6/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9538 - loss: 0.1387
Epoch 7/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9604 - loss: 0.1252
Epoch 8/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9648 - loss: 0.1139
Epoch 9/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9692 - loss: 0.1047
Epoch 10/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9714 - loss: 0.0972
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9912 - loss: 0.0657
Test Accuracy: 0.9912280440330505
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
Predicted Values:
[[0.82361203]
 [0.00179661]
 [0.04449305]
 [0.9768714 ]
 [0.988697  ]]
```

Program 8

Objective: Implement with a program showing Access and manipulation of tensors.

Code:

```
import tensorflow as tf

tensor = tf.constant([[10, 20, 30],
                      [40, 50, 60],
                      [70, 80, 90]])

# Display the original tensor
print("Original Tensor:")
print(tensor)

# Access a single element
print("\nElement at Row 2, Column 3:")
print(tensor[1, 2].numpy())

print("\nFirst Row:")
print(tensor[0].numpy())

print("\nSecond Column:")
print(tensor[:, 1].numpy())

# Slice the tensor
print("\nFirst Two Rows and First Two Columns:")
print(tensor[0:2, 0:2].numpy())

print("\nTensor after Adding 5:")
print((tensor + 5).numpy())

print("\nTensor after Multiplication by 2:")
print((tensor * 2).numpy())

# Reshape the tensor
reshaped = tf.reshape(tensor, (1, 9))
print("\nReshaped Tensor:")
print(reshaped.numpy())
```

Output:

```
Original Tensor:
tf.Tensor(
[[10 20 30]
 [40 50 60]
 [70 80 90]], shape=(3, 3), dtype=int32)

Element at Row 2, Column 3:
60

First Row:
[10 20 30]

Second Column:
[20 50 80]

First Two Rows and First Two Columns:
[[10 20]
 [40 50]]

Tensor after Adding 5:
[[15 25 35]
 [45 55 65]
 [75 85 95]]

Tensor after Multiplication by 2:
[[ 20  40  60]
 [ 80 100 120]
 [140 160 180]]

Reshaped Tensor:
[[10 20 30 40 50 60 70 80 90]]
```

Program 9

Objective: Write a program to show Regression Data Sampling.

Code:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = load_diabetes()

# Features (X) and Target (y)
X = data.data
y = data.target

# Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create the Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Predict the test data
y_pred = model.predict(X_test)

# Print sample sizes
print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

# Print evaluation metrics
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Output:

```
Training Samples: 353
Testing Samples: 89
Mean Squared Error: 2900.193628493482
R2 Score: 0.4526027629719195
```

Program 10

Objective: Write a program to Combat Overfitting.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

iris = load_iris()
X = iris.data
y = iris.target

# Convert target labels to one-hot encoding
y = to_categorical(y)

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the Neural Network
model = Sequential()

# Hidden layer
model.add(Dense(16, activation='relu', input_shape=(4,)))

# Dropout Layer (Combat Overfitting)
model.add(Dropout(0.5))
```

Output:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential m
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.4000 - loss: 1.0890
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5667 - loss: 0.9948
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5500 - loss: 0.9579
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6250 - loss: 0.9210
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6500 - loss: 0.8433
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6833 - loss: 0.8098
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6917 - loss: 0.7208
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6833 - loss: 0.7433
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6750 - loss: 0.6871
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8000 - loss: 0.6081
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 322ms/step - accuracy: 0.8667 - loss: 0.4748
Test Accuracy: 0.8666666746139526
```

Program 11

Objective: Write a program to print Dimensions of dataset.

Code:

```
from sklearn.datasets import load_iris
import pandas as pd

# Load Iris dataset
iris = load_iris()

# Convert to DataFrame
data = pd.DataFrame(iris.data, columns=iris.feature_names)

# Print dimensions
print("Dimensions of the dataset:", data.shape)
print("Number of Rows:", data.shape[0])
print("Number of Columns:", data.shape[1])
```

Output:

```
Dimensions of the dataset: (150, 4)
Number of Rows: 150
Number of Columns: 4
```

Program 12

Objective: Implement with a program showing Access and manipulation of tensors.

Code:

```
import tensorflow as tf

tensor = tf.constant([[10, 20, 30],
                      [40, 50, 60],
                      [70, 80, 90]])

# Display the original tensor
print("Original Tensor:")
print(tensor)

# Access a single element
print("\nElement at Row 2, Column 3:")
print(tensor[1, 2].numpy())

print("\nFirst Row:")
print(tensor[0].numpy())

print("\nSecond Column:")
print(tensor[:, 1].numpy())

# Slice the tensor
print("\nFirst Two Rows and First Two Columns:")
print(tensor[0:2, 0:2].numpy())

print("\nTensor after Adding 5:")
print((tensor + 5).numpy())

print("\nTensor after Multiplication by 2:")
print((tensor * 2).numpy())

# Reshape the tensor
reshaped = tf.reshape(tensor, (1, 9))
print("\nReshaped Tensor:")
print(reshaped.numpy())
```

Output:

```
Original Tensor:
tf.Tensor(
[[10 20 30]
 [40 50 60]
 [70 80 90]], shape=(3, 3), dtype=int32)

Element at Row 2, Column 3:
60

First Row:
[10 20 30]

Second Column:
[20 50 80]

First Two Rows and First Two Columns:
[[10 20]
 [40 50]]

Tensor after Adding 5:
[[15 25 35]
 [45 55 65]
 [75 85 95]]

Tensor after Multiplication by 2:
[[ 20  40  60]
 [ 80 100 120]
 [140 160 180]]

Reshaped Tensor:
[[10 20 30 40 50 60 70 80 90]]
```

Program 13

Objective: Write a program to show Regression Data Sampling.

Code:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = load_diabetes()

# Features (X) and Target (y)
X = data.data
y = data.target

# Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create the Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Predict the test data
y_pred = model.predict(X_test)

# Print sample sizes
print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

# Print evaluation metrics
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Output:

```
Training Samples: 353
Testing Samples: 89
Mean Squared Error: 2900.193628493482
R2 Score: 0.4526027629719195
```

Program 14

Objective: Write a program to Combat Overfitting.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

y = to_categorical(y)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the Neural Network
model = Sequential()
```

```
# Input and Hidden Layer
model.add(Dense(16, activation='relu', input_shape=(4,)))

# Dropout Layer to prevent overfitting
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(3, activation='softmax'))

# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=5)

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

# Display the accuracy
print("Test Accuracy:", accuracy)
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential m
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.2333 - loss: 1.6423
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.3167 - loss: 1.4120
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.2917 - loss: 1.2308
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3333 - loss: 1.1741
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.3833 - loss: 1.0501
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.3833 - loss: 1.0149
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.4917 - loss: 0.8905
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5417 - loss: 0.8281
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.6250 - loss: 0.7839
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5750 - loss: 0.7727
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 368ms/step - accuracy: 0.7333 - loss: 0.5706
Test Accuracy: 0.7333333492279053
```

Program 15

Objective: Write a program to Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

iris = load_iris()
X = iris.data
y = iris.target

y = to_categorical(y)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the ANN
model = Sequential()

# Hidden Layer
model.add(Dense(10, activation='relu', input_shape=(4,)))

# Output Layer
model.add(Dense(3, activation='softmax'))
```

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model (Backpropagation is applied automatically)
model.fit(X_train, y_train, epochs=10, batch_size=5)

# Test the model
loss, accuracy = model.evaluate(X_test, y_test)

# Display the accuracy
print("Test Accuracy:", accuracy)
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.3333 - loss: 1.1645
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.4750 - loss: 1.0560
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5583 - loss: 0.9721
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.6250 - loss: 0.8995
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6833 - loss: 0.8362
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.7833 - loss: 0.7806
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8000 - loss: 0.7280
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8000 - loss: 0.6810
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8333 - loss: 0.6343
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8417 - loss: 0.5941
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 287ms/step - accuracy: 0.9000 - loss: 0.5038
Test Accuracy: 0.8999999761581421
```

Program 16

Objective: Write a program to build a simple autoencoder based on a fully-connected layer in Keras

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# Load the Digits dataset
digits = load_digits()
X = digits.data

# Normalize the data
scaler = MinMaxScaler()
X = scaler.fit_transform(X)

# Split the dataset
X_train, X_test = train_test_split(
    X, test_size=0.2, random_state=42
)

# Input layer
input_layer = Input(shape=(64,))

# Encoder
encoded = Dense(32, activation='relu')(input_layer)

# Decoder
decoded = Dense(64, activation='sigmoid')(encoded)

# Build Autoencoder
autoencoder = Model(input_layer, decoded)
```

```python
# Compile the model
autoencoder.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)

# Train the model
autoencoder.fit(
    X_train,
    X_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, X_test)
)

# Evaluate the model
loss = autoencoder.evaluate(X_test, X_test)

# Print the loss
print("Test Loss:", loss)
```

Output:

```
Epoch 1/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - loss: 0.6124 - val_loss: 0.5106
Epoch 2/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.4523 - val_loss: 0.4196
Epoch 3/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.4081 - val_loss: 0.3955
Epoch 4/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.3850 - val_loss: 0.3740
Epoch 5/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.3641 - val_loss: 0.3555
Epoch 6/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.3467 - val_loss: 0.3404
Epoch 7/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.3329 - val_loss: 0.3289
Epoch 8/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.3224 - val_loss: 0.3202
Epoch 9/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.3144 - val_loss: 0.3135
Epoch 10/10
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.3080 - val_loss: 0.3077
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.3077
Test Loss: 0.3077326714992523
```

Program 17

Objective: Implement Long Short-Term Memory Networks using sample data.

Code:

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

X = np.array([
    [1, 2, 3],
    [2, 3, 4],
    [3, 4, 5],
    [4, 5, 6],
    [5, 6, 7]
])

y = np.array([4, 5, 6, 7, 8])
X = X.reshape((X.shape[0], X.shape[1], 1))

# Build the LSTM model
model = Sequential()

# LSTM layer
model.add(LSTM(50, activation='relu', input_shape=(3, 1)))

# Output layer
model.add(Dense(1))

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Train the model
model.fit(X, y, epochs=10, verbose=1)

test = np.array([[6, 7, 8]])
test = test.reshape((1, 3, 1))
prediction = model.predict(test)
print("Predicted Value:", prediction)
```

Output:

```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step - loss: 39.0363
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step - loss: 38.3870
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 169ms/step - loss: 37.7467
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step - loss: 37.1166
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step - loss: 36.4948
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 170ms/step - loss: 35.8798
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step - loss: 35.2690
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 159ms/step - loss: 34.6619
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step - loss: 34.0581
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - loss: 33.4572
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 383ms/step
Predicted Value: [[0.9749566]]
```

Program 18

Objective: Write a program showing Automatic Image Captioning with Keras ---Facial Recognition.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize images
X_train = X_train / 255.0
X_test = X_test / 255.0

# Build the model
model = Sequential()

# Flatten image
model.add(Flatten(input_shape=(28, 28)))

# Hidden layer
model.add(Dense(128, activation='relu'))

# Output layer
model.add(Dense(10, activation='softmax'))

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=5)

_, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequen
  super().__init__(**kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9254 - loss: 0.2616
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9661 - loss: 0.1155
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9757 - loss: 0.0798
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9821 - loss: 0.0581
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9850 - loss: 0.0466
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9764 - loss: 0.0749
Test Accuracy: 0.9764000177383423
```

Program 19

Objective: Write a program to implement Gradient Descent Algorithm for optimization.

Code:

```
# Function: f(x) = x^2
# Derivative: f'(x) = 2x

# Initial value of x
x = 10

# Learning rate
learning_rate = 0.1

epochs = 10

# Gradient Descent
for i in range(epochs):

    gradient = 2 * x
    x = x - learning_rate * gradient
    print("Iteration", i+1, ": x =", x)

# Print optimized value
print("\nOptimized x =", x)
print("Minimum value of f(x) =", x**2)
```

Output:

```
Iteration 1 : x = 8.0
Iteration 2 : x = 6.4
Iteration 3 : x = 5.12
Iteration 4 : x = 4.096
Iteration 5 : x = 3.2768
Iteration 6 : x = 2.62144
Iteration 7 : x = 2.0971520000000003
Iteration 8 : x = 1.6777216000000004
Iteration 9 : x = 1.3421772800000003
Iteration 10 : x = 1.0737418240000003

Optimized x = 1.0737418240000003
Minimum value of f(x) = 1.1529215046068475
```

Program 20

Objective: Write a program to demonstrate Activation Functions (ReLU, Sigmoid, Tanh).

Code:

```
import tensorflow as tf

x = tf.constant([-2.0, -1.0, 0.0, 1.0, 2.0])

# ReLU Activation Function
relu = tf.nn.relu(x)

# Sigmoid Activation Function
sigmoid = tf.nn.sigmoid(x)

# Tanh Activation Function
tanh = tf.nn.tanh(x)

# Display results
print("Input Values:")
print(x.numpy())

print("\nReLU Output:")
print(relu.numpy())

print("\nSigmoid Output:")
print(sigmoid.numpy())

print("\nTanh Output:")
print(tanh.numpy())
```

Output:

```
Input Values:
[-2. -1.  0.  1.  2.]

ReLU Output:
[0. 0. 0. 1. 2.]

Sigmoid Output:
[0.11920292 0.26894143 0.5        0.7310586  0.8807971 ]

Tanh Output:
[-0.9640276 -0.7615942  0.         0.7615942  0.9640276]
```

Program 21

Objective: Write a program to implement a Multilayer Perceptron (MLP) using Keras.

Code:

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

model.fit(x_train,y_train,epochs=10,batch_size=32,validation_split=0.1)

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

Output:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequen
  super().__init__(**kwargs)
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9239 - loss: 0.2606 - val_accuracy: 0.9678 - val_loss: 0.1119
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9669 - loss: 0.1095 - val_accuracy: 0.9718 - val_loss: 0.0902
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9766 - loss: 0.0748 - val_accuracy: 0.9775 - val_loss: 0.0807
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9827 - loss: 0.0556 - val_accuracy: 0.9782 - val_loss: 0.0779
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9862 - loss: 0.0426 - val_accuracy: 0.9777 - val_loss: 0.0801
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9873 - loss: 0.0367 - val_accuracy: 0.9813 - val_loss: 0.0722
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9913 - loss: 0.0270 - val_accuracy: 0.9773 - val_loss: 0.0898
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9912 - loss: 0.0262 - val_accuracy: 0.9805 - val_loss: 0.0782
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9931 - loss: 0.0199 - val_accuracy: 0.9775 - val_loss: 0.0926
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9935 - loss: 0.0186 - val_accuracy: 0.9768 - val_loss: 0.1072
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9721 - loss: 0.1250
Test Loss: 0.12497216463088989
Test Accuracy: 0.972100019454956
```

Program 22

Objective: Write a program to perform Image Classification using CNN (Convolutional Neural Network).

Code:

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize images
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape for CNN (28x28x1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Build CNN model
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Display model summary
model.summary()

# Train model
model.fit(x_train, y_train, epochs=5, batch_size=64)

# Evaluate model
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", accuracy * 100)

# Predict one image
index = 15  # Change index to test different digits

plt.imshow(x_test[index].reshape(28, 28), cmap='gray')
plt.title("Handwritten Digit")
plt.axis('off')
plt.show()

prediction = model.predict(x_test[index:index+1], verbose=0)
predicted_digit = np.argmax(prediction)

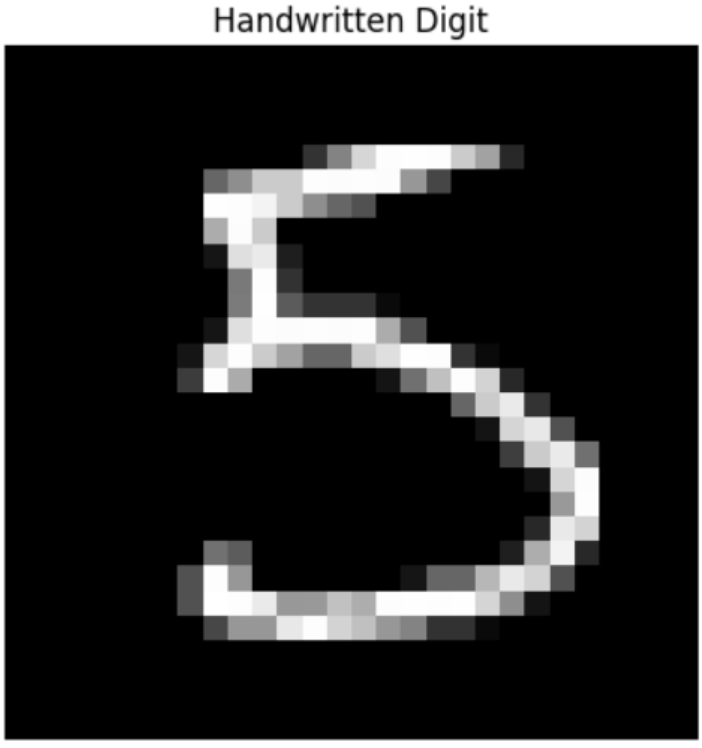
print("Actual Digit    :", y_test[index])
print("Predicted Digit :", predicted_digit)
print("Confidence      : {:.2f}%".format(np.max(prediction) * 100))
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When usir
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 59s 59ms/step - accuracy: 0.9541 - loss: 0.1533
Epoch 2/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 46s 49ms/step - accuracy: 0.9861 - loss: 0.0451
Epoch 3/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 45s 48ms/step - accuracy: 0.9908 - loss: 0.0302
Epoch 4/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 46s 49ms/step - accuracy: 0.9926 - loss: 0.0229
Epoch 5/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 44s 47ms/step - accuracy: 0.9944 - loss: 0.0171
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9888 - loss: 0.0309
Test Accuracy: 98.87999892234802
```




```
Actual Digit    : 5
Predicted Digit : 5
Confidence      : 100.00%
```

Program 23

Objective: Write a program to apply Droupout Regularization in neural network.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

iris = load_iris()
X = iris.data
y = iris.target

y = to_categorical(y)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the Neural Network
model = Sequential()

# Input + Hidden Layer
model.add(Dense(16, activation='relu', input_shape=(4,)))
```

```
# Dropout Layer
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(3, activation='softmax'))

# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=5)

# Evaluate the model
_, accuracy = model.evaluate(X_test, y_test)

# Print accuracy
print("Test Accuracy:", accuracy)
```

Output:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential m
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.2583 - loss: 1.3697
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.3083 - loss: 1.2797
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3250 - loss: 1.1554
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4583 - loss: 1.0493
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.4667 - loss: 1.0072
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6167 - loss: 0.7806
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5833 - loss: 0.7937
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6250 - loss: 0.8077
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.6167 - loss: 0.7698
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.6333 - loss: 0.7194
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 375ms/step - accuracy: 0.7333 - loss: 0.5345
Test Accuracy: 0.7333333492279053
```

Program 24

Objective: Write a program to implement Batch Normalization.

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

iris = load_iris()
X = iris.data
y = iris.target

y = to_categorical(y)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize the input features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build the Neural Network
model = Sequential()

# Hidden Layer
model.add(Dense(16, activation='relu', input_shape=(4,)))
```

```
# Batch Normalization Layer
model.add(BatchNormalization())

# Output Layer
model.add(Dense(3, activation='softmax'))

# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=5)

# Evaluate the model
_, accuracy = model.evaluate(X_test, y_test)

# Print Accuracy
print("Test Accuracy:", accuracy)
```

Output:

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential m
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.2250 - loss: 1.4909
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4500 - loss: 1.0683
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6167 - loss: 0.8164
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7583 - loss: 0.6696
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7000 - loss: 0.6862
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7583 - loss: 0.5745
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8083 - loss: 0.5545
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8333 - loss: 0.4788
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7500 - loss: 0.5375
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8333 - loss: 0.4627
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 301ms/step - accuracy: 0.9333 - loss: 0.3921
Test Accuracy: 0.9333333373069763
```

Program 25

Objective: Write a program to perform Sentiment Analysis using Deep Learning (LSTM).

Code:

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load IMDB dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=5000)

X_train = pad_sequences(X_train, maxlen=200)
X_test = pad_sequences(X_test, maxlen=200)

# Build the LSTM model
model = Sequential()

# Embedding layer
model.add(Embedding(input_dim=5000, output_dim=64, input_length=200))

# LSTM layer
model.add(LSTM(64))

# Output layer
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.fit(X_train, y_train, epochs=3, batch_size=64)

# Evaluate the model
_, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
391/391 ━━━━━━━━━━━━━━━━━━━━ 61s 149ms/step - accuracy: 0.7975 - loss: 0.4320
Epoch 2/3
391/391 ━━━━━━━━━━━━━━━━━━━━ 85s 155ms/step - accuracy: 0.8888 - loss: 0.2788
Epoch 3/3
391/391 ━━━━━━━━━━━━━━━━━━━━ 79s 148ms/step - accuracy: 0.9106 - loss: 0.2288
782/782 ━━━━━━━━━━━━━━━━━━━━ 24s 31ms/step - accuracy: 0.8707 - loss: 0.3270
Test Accuracy: 0.8706799745559692
```

Program 26

Objective: Write a program to implement Recurrent Neural Network (RNN) for sequence prediction.

Code:

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

X = np.array([
    [1, 2, 3],
    [2, 3, 4],
    [3, 4, 5],
    [4, 5, 6],
    [5, 6, 7]
])

# Target values
y = np.array([4, 5, 6, 7, 8])

# Reshape input to (samples, time steps, features)
X = X.reshape((X.shape[0], X.shape[1], 1))

# Build the RNN model
model = Sequential()

# RNN Layer
model.add(SimpleRNN(50, activation='relu', input_shape=(3, 1)))

# Output Layer
model.add(Dense(1))
```

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='mse'
)

# Train the model
model.fit(X, y, epochs=10)

# Test data
test = np.array([[6, 7, 8]])
test = test.reshape((1, 3, 1))

# Predict the next value
prediction = model.predict(test)

print("Predicted Value:", prediction)
```

Output:

```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - loss: 29.0263
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 27.1756
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 25.4418
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 23.7907
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 22.2340
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 20.7386
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step - loss: 19.3000
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 17.9189
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - loss: 16.5926
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 15.3177
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 170ms/step
Predicted Value: [[3.85815]]
```